A Domain-Specific Language for Geometric Simplicial Complexes working with PolyLogicA

Simone Riccio

July 16, 2026

Abstract

This report presents a Domain-Specific Language (DSL) and a web interface designed for defining and visualizing geometric simplicial complexes in the Euclidean plane. The DSL allows users to instantiate points, define simplices, and apply geometric transformations and topological operations. The generated complexes are then serialized into a poset representation suitable for use in PolyLogicA, a spatial model checker. The web interface provides an interactive 3D visualization of the defined complexes, enabling users to explore their spatial properties and perform spatial logic queries.

1 Introduction

Spatial logics are extensions of classical modal logic to reason rigorously about topological and geometrical properties of spaces. The VoxLogicA project [3] provides a framework for spatial model checking, allowing users to verify properties of topological spaces represented as closure spaces. Beyond theoretical explorations, spatial logics and spatial model checkers have found highly successful practical applications in computer vision, most notably in medical imaging analysis for brain tissue segmentation [5].

Specifically, the PolyLogicA branch of VoxLogicA focuses on reasoning about posets. The project introduces a novel Domain-Specific Language (DSL) and an interactive web interface, designed specifically for the generation and manipulation of geometric simplicial complexes. The proposed DSL can instantiate points, define simplices, and apply both geometric transformations (such as translation and scaling) and topological operations (such as boundaries and stars).

Before the creation of the DSL, developers working with PolyLogicA had to manually write JSON files representing posets in a non-human-intuitive format. This process was error-prone and slow, especially for complex geometric structures. The DSL abstracts this complexity, allowing users to define geometric simplicial complexes in a more natural and mathematically rigorous manner.

Furthermore, spatial reasoning is inherently visual. The project includes a web interface providing a 3D scene rendered using Three.js, allowing users to visualize the geometric simplicial complexes they define. This visual feedback is crucial for understanding the spatial relationships and properties of the complexes. The interface also allows users to input spatial logic queries and visualize them.

The remainder of this report is structured as follows: Section 2 provides the theoretical background, Section 3 details the design and implementation of the DSL, Section 4 describes the web interface, and Section 5 presents examples and some query results.

2 Theoretical Background

2.1 Geometry and Simplicial Complexes

Definition 1 (Simplex)**.** *Given a set of $k+1$ affinely independent points in $\mathbb{R}^n$ denoted as $\{v_0, v_1, \ldots, v_k\}$, the k-simplex σ spanned by these points is defined as the convex hull:*

$$\sigma = [v_0, v_1, \ldots, v_k] = \left\{ \sum \lambda_i v_i \mid \sum \lambda_i = 1, \lambda_i \geq 0 \right\}$$

Definition 2 (Simplicial Complex)**.** *A simplicial complex K is a finite collection of simplices such that:*

1. *Every face of a simplex in K is also in K.*
2. *The intersection of any two simplices in K is either empty or a face of both simplices.*

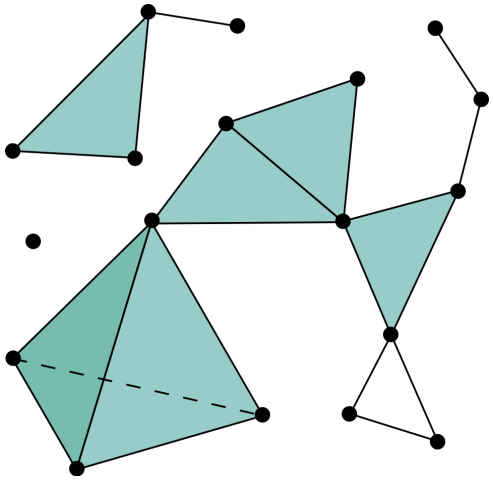

Figure 1: A simplicial complex.

Remark 1. *Simplicial complexes provide a combinatorial representation of topological spaces, allowing for the application of algebraic topology techniques and spatial logic reasoning.*

Definition 3 (Abstract Simplicial Complex)**.** *An abstract simplicial complex is a set K of finite subsets of a vertex set V such that if $\sigma \in K$ and $\tau \subseteq \sigma$, then $\tau \in K$. The elements of K are called simplices, and the elements of V are called vertices.*

Definition 4 (Dimension)**.** *The dimension of a simplex $\sigma = [v_0, v_1, \ldots, v_k]$ is defined as k. The dimension of a simplicial complex K is the maximum dimension of its simplices:*

Definition 5 (Downward Closure)**.** *The downward closure of a simplex σ is the set of all its faces:*

$$C(\sigma) = \{\tau \subseteq \sigma \mid \tau \text{ is a face of } \sigma\}.$$

2.2 Posets

Definition 6 (Partially Ordered Set (Poset))**.** *A partially ordered set (poset) is a set P equipped with a binary relation $\leq$ that is reflexive, antisymmetric, and transitive.*

Definition 7 (Hasse Diagram)**.** *A Hasse diagram is a graphical representation of a finite poset, where each element is represented as a vertex, and edges are drawn to indicate the covering relations. An edge from vertex a to vertex b is drawn if $a < b$ and there is no c such that $a < c < b$. The diagram is typically drawn so that if $a < b$, then b appears higher than a in the diagram.*

Remark 2. *The set of simplices in a simplicial complex, ordered by inclusion, forms a poset. The Hasse diagram of this poset visually represents the inclusion relationships among the simplices, which is crucial for spatial model checking in PolyLogicA.*

Remark 3. *The transition from geometric simplicial complexes to partially ordered sets (posets) is the core computational step for interfacing with spatial model checkers like PolyLogicA. While a simplex is defined by its coordinates in Euclidean space, its logical existence in the model checker is defined by its position in a hierarchy of inclusion.*

2.3 Spatial Logic

Broadly defined, a spatial logic is any formal language interpreted over a class of structures featuring geometrical entities and relations. [1] provides a comprehensive overview of the various spatial logics, their semantics, and applications.

The origin of this field lies in the topological approach to modal logic initiated by Alfred Tarski, who recognized that modal operators could represent topological notions such as interior and closure.

Modern developments in spatial logic extend these ideas to discrete spatial structures (like graphs or images) using a generalized topological space called a closure space [2].

Definition 8 (Closure Space)**.** *A closure space is a pair $(X, \mathcal{C})$ where X is a set and $\mathcal{C} : \mathcal{P}(X) \to \mathcal{P}(X)$ is a closure operator satisfying the following properties for all $A, B \subseteq X$:*

1. $\mathcal{C}(\emptyset) = \emptyset$
2. $A \subseteq \mathcal{C}(A)$ *(Extensivity)*
3. *If $A, B \subseteq X$ and $\mathcal{C}(A \cup B) = \mathcal{C}(A) \cup \mathcal{C}(B)$*

Remark 4. *In any topological space X we can define $\mathcal{C}(A)$ as the topological closure of A, which is the smallest closed set containing A. This closure operator satisfies the properties of a closure operator, making $(X, \mathcal{C})$ a closure space.*

The closure operator in this case has one additional property: it is idempotent, meaning that $\mathcal{C}(\mathcal{C}(A)) = \mathcal{C}(A)$ for all $A \subseteq X$. This property is not required for a closure space, but it is a defining characteristic of topological spaces.

Example 1 (A closure space which is not a topological space)**.** *Consider for an oriented graph $G = (V, E)$ the closure operator $\mathcal{C}$ defined as follows: for any $A \subseteq V$, $\mathcal{C}(A)$ is the set of all vertices in V that can be reached from any vertex in A by a directed path in G. This closure operator satisfies the properties of a closure operator, making $(V, \mathcal{C})$ a closure space.*

However, this closure operator is not necessarily idempotent. For example, if A consists of a single vertex with outgoing edges to other vertices, then $\mathcal{C}(A)$ will include those reachable vertices. Applying $\mathcal{C}$ again may yield additional vertices that are reachable from the newly included vertices, thus $\mathcal{C}(\mathcal{C}(A))$ may be strictly larger than $\mathcal{C}(A)$. Therefore, $(V, \mathcal{C})$ is a closure space but not a topological space.

If the graph is undirected, then the closure operator becomes idempotent, and $(V, \mathcal{C})$ is a topological space.

Ciancia et al. define a spatial logic for closure spaces for reasoning about spatial properties of closure spaces.

Definition 9 (Spatial Logic for Closure Spaces [2])**.** *In a closure space (X, C), the syntax of the spatial logic is defined as follows:*

$$\phi ::= p \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \mathcal{N}\phi \mid \phi_1 \mathcal{U} \phi_2$$

where p is a propositional variable, $\neg$ is negation, $\wedge$ is conjunction, $\mathcal{N}$ is the "next" operator, and $\mathcal{U}$ is the "until" operator. The semantics of the logic is defined in terms of the closure operator $\mathcal{C}$ and the satisfaction relation $\models$ as follows:

- $(X, \mathcal{C}), x \models p$ *if* $x \in V(p)$*, where* $V(p)$ *is the set of points in* X *where* p *is true.*
- $(X, \mathcal{C}), x \models \neg\phi$ *if* $(X, \mathcal{C}), x \not\models \phi$*.*
- $(X, \mathcal{C}), x \models \phi_1 \wedge \phi_2$ *if* $(X, \mathcal{C}), x \models \phi_1$ *and* $(X, \mathcal{C}), x \models \phi_2$*.*
- $(X, \mathcal{C}), x \models \mathcal{N}\phi$ *if there exists* $y \in \mathcal{C}(\{x\})$ *such that* $(X, \mathcal{C}), y \models \phi$*.*
- $(X, \mathcal{C}), x \models \phi_1 \mathcal{U} \phi_2$ *if there exists a finite sequence of points* $x_0, x_1, \ldots, x_n \in X$ *such that* $x_0 = x$*,* $(X, \mathcal{C}), x_n \models \phi_2$*, and for all* $0 \leq i < n$*,* $(X, \mathcal{C}), x_i \models \phi_1$*.*

We will not go into the details of the semantics of the logic, but we will focus on how to represent geometric simplicial complexes in a way that allows us to reason about their spatial properties using this logic.

3 Geometric DSL

3.1 Structure

The DSL is designed to be a simple and expressive language for defining geometric simplicial complexes. The DSL's code is structured into several components, each responsible for a specific aspect of the language's functionality. The main components are:

- **Parser**: The parser is responsible for reading the DSL code and converting it into an abstract syntax tree (AST).
- **Core**: The core component is responsible for evaluating the AST and constructing the corresponding geometric simplicial complex.
- **Exporter**: The exporter component is responsible for serializing the geometric simplicial complex into a format that can be used by the web interface and by PolyLogicA.

They are implemented in the following files:

```
project-root/
├── src/
│   ├── parser.py
│   ├── core.py
│   └── exporter.py
├── web/
│   ├── index.html
│   ├── render.js
│   └── visualization.js
└── server.py
```

3.2 Syntax

The syntax of the DSL is designed for defining geometric simplicial complexes in a concise manner. The language allows users to specify vertices, simplices, and operations on them.

```
    program: statement*

    ?statement: point_decl
              | complex_decl
              | assign
              | render_stmt
              | func_decl
              | return_stmt

    point_decl: "point" IDENT "=" tuple_coord
    complex_decl: "complex" IDENT "=" expr
    render_stmt: "render" IDENT
    func_decl: "function" IDENT "(" param_list ")" "{{" statement* "}}"
    return_stmt: "return" expr

    ?expr: op_call
        | func_call
        | IDENT
        | vertices_list
        | tuple_coord
        | NUMBER
        | "(" expr ")"

    op_call: OP "(" arg_list? ")"
    func_call: expr "(" call_args ")"

    tuple_coord: "(" expr ("," expr)+ ")"
    vertices_list: "[" id_list "]"
    id_list: IDENT ("," IDENT)*
    arg_list: expr ("," expr)*
    param_list: (IDENT ("," IDENT)*)?
    call_args: (expr ("," expr)*)?

    OP: /\b{op_regex}\b/

    IDENT: /[A-Za-z_][A-Za-z0-9_]*/

    NUMBER: /-?[0-9]+(\.[0-9]+)?/

    COMMENT: "//" /[^\n]/* "\n"
    %ignore COMMENT
    WS: /[ \t\f\r\n]+/
    %ignore WS
```

Listing 1: Grammar of the Geometric DSL

To parse this grammar, we use the Lark parser library in Python in the file `dsl_parser.py`. The parser generates an abstract syntax tree (AST) that represents the structure of the input program. The AST is then traversed to evaluate the expressions and construct the corresponding geometric simplicial complex.

- **Point Declarations (`PointDecl`)**: Instantiates a coordinates in $\mathbb{R}^n$ as tuples of numbers.
- **Complex Declarations (`ComplexDecl`)**: Instantiates a simplicial complex from a list of vertices or through operations.
- **Render Statements (`RenderStmt`)**: Registers a target complex to be exported and visualized by the web interface.
- **Function Definitions (`FunctionDecl`)**: Standard functional structures that yield runtime closures to enable procedural abstraction.

3.3 Semantics: Evaluation and Denotational Values

The evaluation of the program is defined over well-defined expressible and denotational values:

```
type EVal = Geometric | Num | Closure | None
type DVal = EVal | Operator | Closure

type Geometric = Point | Complex
type Num = int | float
```

Listing 2: Types for Evaluation

The variable evaluation environment is a mapping from identifiers to denotational values:

$$\rho : \text{IDENT} \to \text{DVal}$$

It is implemented as a Dictionary in Python, where the keys are the identifiers and the values are the corresponding denotational values.

```
type Environment = Dict[str, DVal]
```

Listing 3: Environment

The evaluation of the program is defined as a function that takes an AST and returns an environment.

3.4 Semantics: Operations

The language supports a set of operations that can be applied to geometric simplicial complexes. These operations are defined as functions that take complexes and numbers as input and return a new complex as output. The operations include:

```
def union(complex1: Complex, complex2: Complex) -> Complex:

def intersection(complex1: Complex, complex2: Complex) -> Complex:

def difference(complex1: Complex, complex2: Complex) -> Complex:

def translate(complex: Complex, vector: Tuple[float, float]) ->
Complex:

def scale(complex: Complex, factor: float) -> Complex:
```

Listing 4: Geometric Operations

The semantics of these operations are defined in terms of the underlying geometric representation, therefore we will not go into the details making the assumption that the reader is familiar with basic geometric operations.

There are also some other operations which may have more complex semantics, such as:

```
    def boundary(complex: Complex) -> Complex:
        """Returns the boundary of the complex"""

    def star(complex: Complex, vertex: Point) -> Complex:
        """Returns the star of a vertex in the complex"""
```

Listing 5: Topological Operations

- **Boundary (∂):** For a simplicial complex K, the boundary operator computes its topological boundary. Geometrically, if K is a d-dimensional simplicial complex, its boundary consists of those $(d-1)$-simplices that are faces of exactly one d-simplex, along with their downward closure:

  $$\partial(K) = \{\sigma \in K \mid \dim(\sigma) < \dim(K) \text{ and } \sigma \text{ is a face of an odd number of maximal simplices}\}.$$

  In our constructive implementation, it extracts this sub-complex, preserving the combinatorial and structural requirements of a simplicial complex.

- **Star (St):** Given a simplicial complex K and a target vertex (or sub-complex) v, the star consists of all simplices in K that contain v:

  $$\mathrm{St}(v) = \{\sigma \in K \mid v \subseteq \sigma\}.$$

  Since the open star is not necessarily closed under taking subset faces, our DSL implementation returns the **closed star** (the closure of the star), which automatically includes all subset faces, guaranteeing that the result forms a valid, self-contained sub-complex.

These have been implemented because one of the future goals of this project would be to reason about algebraic topological concepts such as holes.

3.5 Semantics: Render Operation

Another operation is the `render` operation, which is used to mark a complex for export and visualization. The semantics of this operation is to add the complex to a list of render targets, which will be serialized and sent to the web interface for visualization.

4 Implementation Details

The execution pipeline of the DSL consists of three main phases: parsing the source text into an Abstract Syntax Tree (AST), dynamically evaluating the AST within a scoped environment, and serializing the resulting geometric structures into a partially ordered set (poset) for PolyLogicA.

4.1 Core Data Structures

To guarantee mathematical correctness, especially regarding the properties of simplicial complexes (where the order of vertices does not matter and duplicates are not allowed), we leverage Python's native data structures.

In `core.py`, a `Point` is defined as a dataclass, ensuring it is hashable. A `Complex` is represented as a Python `Set` of `FrozenSet`s of Points.

This specific formulation guarantees that operations like union and intersection can directly leverage highly optimized set-theoretic operations at the Python interpreter level (6).

```python
@dataclass(frozen=True)
class Point:
    coords: Tuple[float, ...]

@dataclass
class Complex:
    simplices: Set[FrozenSet[Point]]
```

Listing 6: Core Geometric Data Structures

4.2 AST Evaluation and Control Flow

The evaluator traverses the AST nodes defined in `parser.py` (e.g., `PointDecl`, `ComplexDecl`, `FuncCall`) and recursively resolves them against the active `Environment`.

A notable implementation detail is the handling of function calls and lexical closures. When a `FunctionDecl` is evaluated, it is packed into a `Closure` object that captures the current environment. To manage control flow—specifically early exits via `return` statements inside function bodies—we utilize a custom exception mechanism(7):

```python
class ReturnException(Exception):
    def __init__(self, value: EVal):
        self.value = value
```

Listing 7: Exception-based Control Flow for Returns

When the evaluator encounters a `ReturnStmt`, it raises a `ReturnException` containing the evaluated expression. The function application wrapper catches this exception and extracts the value, cleanly simulating call-stack unwinding.

4.3 Topological Serialization and Poset Generation

The transition from continuous geometric representations to discrete logical structures occurs in the `export.py` module. PolyLogicA requires topological spaces to be represented as Hasse diagrams (posets) of closure spaces.

To achieve this, we used a `_build_downward_closure` algorithm. For every complex marked by a `render_targets` directive, the algorithm extracts all maximal simplices and systematically generates all possible sub-simplices using Python's `itertools.combinations` (8).

```python
def _build_downward_closure(complexes: ComplexEnv) -> tuple[list[
    frozenset], dict[frozenset, str], dict[frozenset, set[str]]]:
    """Generates all sub-simplices, assigning unique string IDs and
    tracking their source variable atoms."""

    for name, complex_obj in complexes.items():
        for max_simplex in complex_obj.simplices:
            pts = list(max_simplex)
            # Generate all dimensional subsets (vertices, edges, faces)
            for r in range(1, len(pts) + 1):
                for sub_combo in itertools.combinations(pts, r):
                    simplex_atoms[frozenset(sub_combo)].add(name)

    sorted_simplices = sorted(simplex_atoms.keys(), key=len)
    simplex_to_id = {simp: str(idx) for idx, simp in enumerate(
    sorted_simplices)}
    return sorted_simplices, simplex_to_id, simplex_atoms
```

Listing 8: Downward Closure Generation

4.4 Integration with PolyLogicA

The serialized poset is then formatted into a JSON structure compatible with PolyLogicA's input requirements. Each simplex is assigned a unique identifier, and the covering relations are encoded as adjacency lists. This allows the spatial model checker to interpret the geometric structures and evaluate spatial logic queries effectively.

The function `export_to_polylogica` in `export.py` handles this final step, ensuring that the output is both syntactically and semantically valid for PolyLogicA's processing pipeline.

5 Web Interface

To test the DSL, I implemented a web interface using Three.js, which allows for the visualization of the geometric simplicial complexes defined in the DSL. The web interface is implemented in the `web` directory, and consists of an HTML file (`index.html`) and two JavaScript files (`render.js` and `visualization.js`).

The UI is designed to be simple and intuitive, as seen in Figure 2.

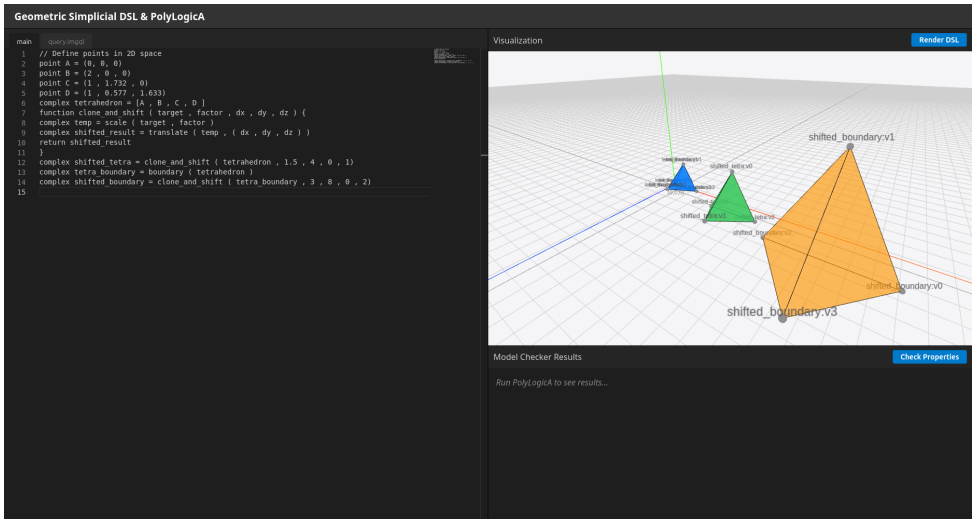

Figure 2: Web Interface for Visualizing Geometric Simplicial Complexes

The interface is divided in half: on the left side there are two selectable text areas, one for the DSL code and one for the spatial logic queries, and on the right side there is a 3D visualization of the geometric simplicial complexes defined in the DSL code.

In the bottom right corner there are the results of the spatial logic queries, which are displayed as a list of simplices that satisfy the query. The user can select a property of the results by clicking the "Draw Results" button, and the simplices that satisfy the query will be highlighted in the 3D visualization.

5.1 3D and higher-dimensional

We can define also 3D simplicial complexes using the DSL, and the web interface will render them in 3D. The code is similar and intuitive, for example in Figure 2

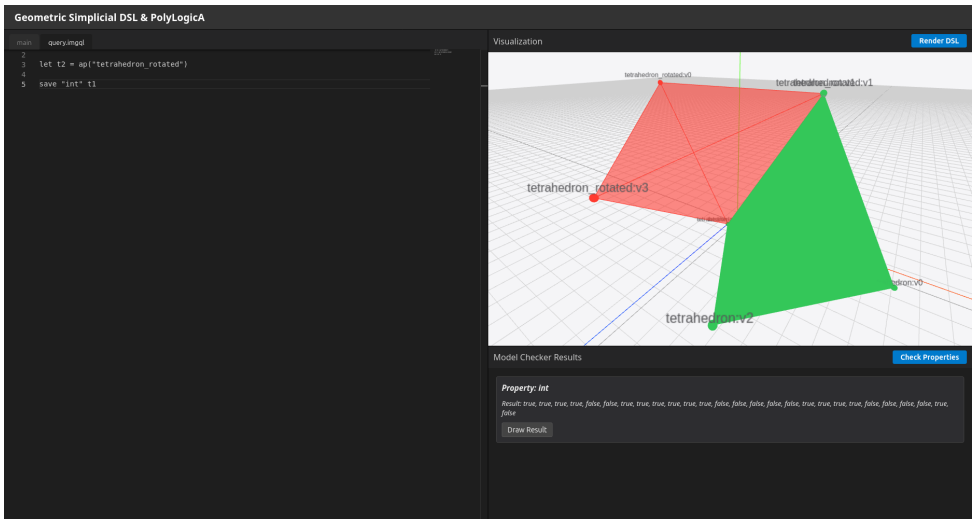

Figure 3: Web Interface for Visualizing Spatial Logic Query Results

```
point A = (0, 0, 0)
point B = (2, 0, 0)
point C = (1, 1.732, 0)
point D = (1, 0.577, 1.633)
complex tetrahedron = [A, B, C, D]

function clone_and_shift(target, factor, dx, dy, dz) {
    complex temp = scale(target, factor)
    complex shifted_result = translate(temp, (dx, dy, dz))
    return shifted_result
}

complex shifted_tetra = clone_and_shift(tetrahedron, 1.5, 4, 0, 1)
complex tetra_boundary = boundary(tetrahedron)
complex shifted_boundary = clone_and_shift(tetra_boundary, 3, 8, 0, 2)
```

Listing 9: DSL Code for 3D Simplicial Complex

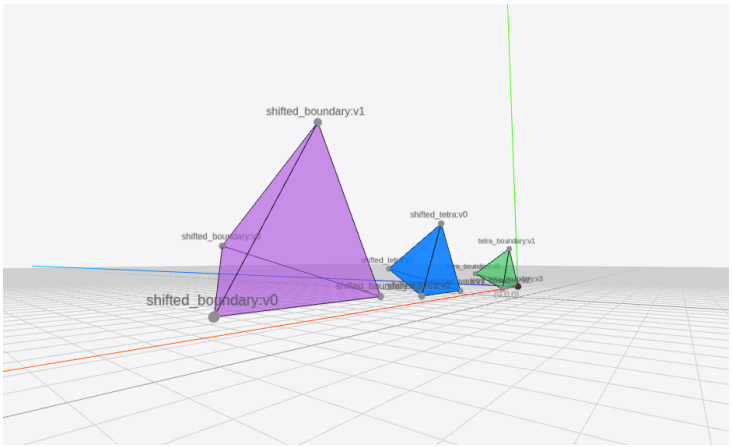

Figure 4: Web Interface for Visualizing 3D Simplicial Complexes

Furthermore, despite not being able to visualize higher-dimensional simplicial complexes, we can still define them using the DSL and export them to PolyLogicA for spatial logic reasoning. For example, we can define a 4D simplex as follows:

```
point A = (0, 0, 0, 0)
point B = (2, 0, 0, 0)
point C = (1, 1.732, 0, 0)
point D = (1, 0.577, 1.633, 0)
point E = (1, 0.577, 0.408, 1.414)

complex pentachoron = scale([A, B, C, D, E], 7)
complex bound = boundary(boundary(pentachoron))
render bound
```

Listing 10: DSL Code for 4D Simplicial Complex

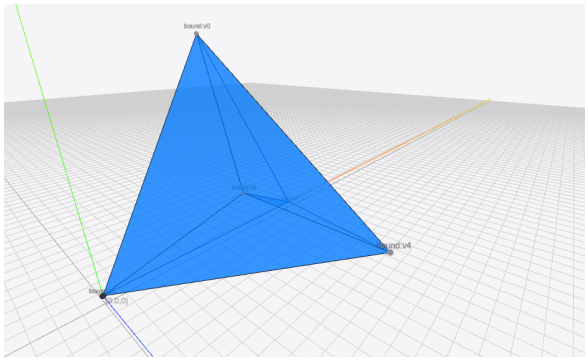

Figure 5: Pentachoron boundary

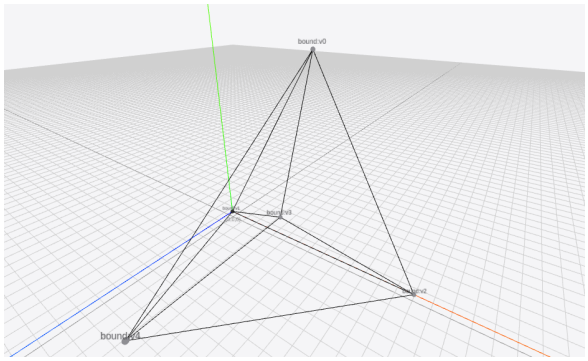

Figure 6: Boundary of Pentachoron boundary

6 Examples and Model Checking

We define a simple 2D simplicial complex representing three triangles with some of their edges shared. The DSL code snippet is as follows:

```
point A = (0, 0, 0)
point B = (2 , 0 , 0)
point C = (1 , 1.732 , 0)
point D = (1 , 0.577 , 1.633)

complex tetrahedron = translate(scale([A , B , C , D ], 4), (0, 0, 1))
complex tetrahedron_rotated = rotate(tetrahedron, 60)

```

Listing 11: DSL Code for Example 1

This will generate the simplicial complex seen in Figure 7. The downward closure of this complex will include all vertices, edges, and faces, which will be serialized into a poset for PolyLogicA. We can then apply spatial logic queries to this complex using PolyLogicA. For example:

```
let t1 = ap("tetrahedron")
let t2 = ap("tetrahedron_rotated")
let intersection = t1 & t2
save "int" intersection
```

Listing 12: Query for Example 1

And the result of this query is shown in Figure 8. The intersection of the two complexes is a single triangle, which is highlighted in green in the figure.

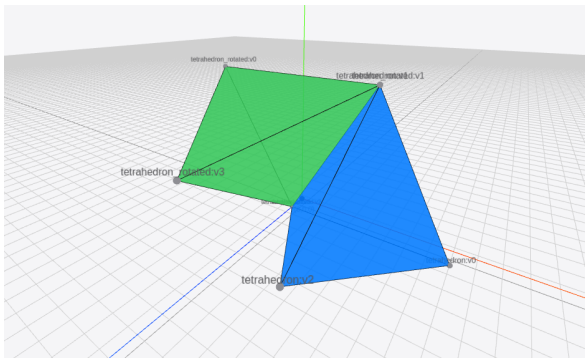

Figure 7: Result of the rendering of Example 1

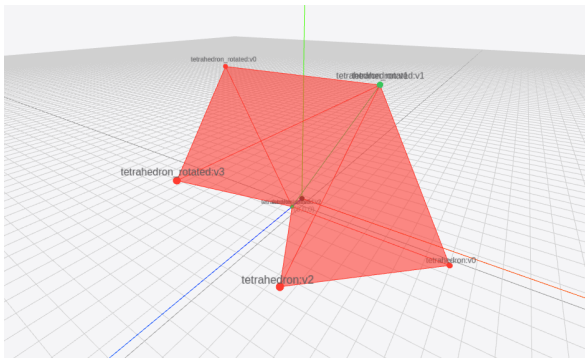

Figure 8: Result of the Query for Example 1

We can also define more complex objects using functions and operations. For example:

```
point p0 = (0, 0, 0)
point p1 = (2, 0, 0)
point p2 = (1, 1.7, 0)
point p3 = (1, 0.5, 1.6)
complex t1 = scale([p0, p1, p2, p3], 4)

function shift (dx ,dy ) {
point offset = (dx ,dy )
    return translate(t1 , offset )
}
complex t2 = shift(10, 0)
complex t3 = shift(20, 0)
complex t4 = shift(10, 15)
complex together = union (t1 , union(t2 , union(t3 , t4 )))

```

Listing 13: DSL Code for Example 2

The resulting complex is shown in Figure 9. It consists of four triangles, with some of them sharing edges and vertices.

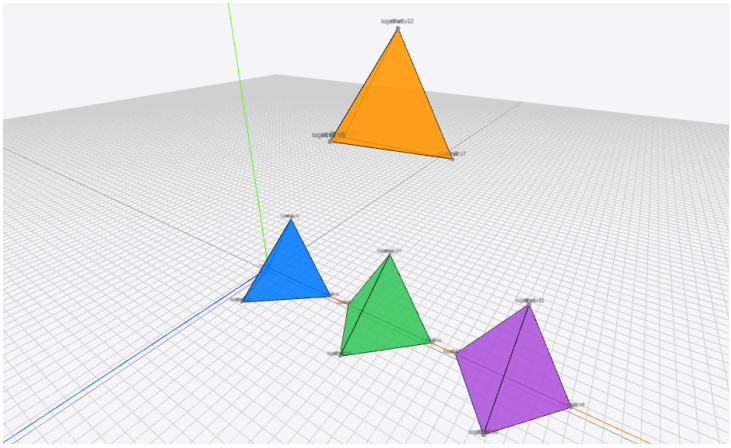

Figure 9: Resulting Complex from Example 2

And we can apply spatial logic queries to this complex to check for connected components by using the $\mathcal{U}$ operator, which computes the closure of a complex with respect to another complex.

For example, we can check if *together* is connected to $t2$ by computing the closure of *together* with respect to $t2$:

```
let t1 = ap("t1")
let t2 = ap("t2")
let t3 = ap("t3")
let t4 = ap("t4")

let together = ap("together")
let near = gamma(together, t2)
save "near" near

```

Listing 14: Query for Example 2

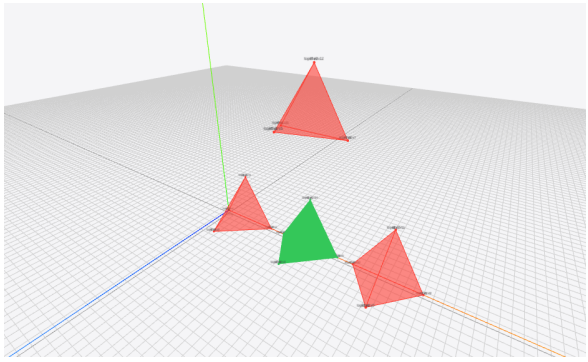

Figure 10: Result of the Query for Example 2 with scale factor = 4

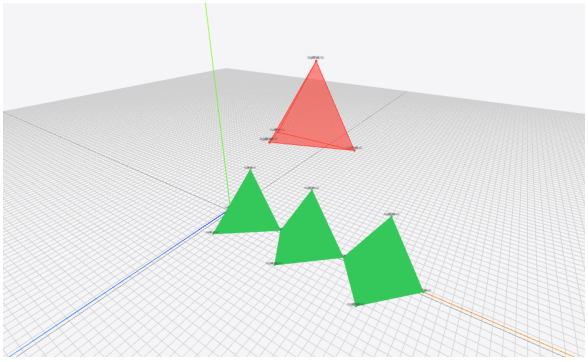

Figure 11: Result of the Query for Example 2 with scale factor = 5

Remark 5. *In this project I am treating the geometric simplicial complexes as formulas. In a future version of the DSL, I would like to treat them only as models that can have different assigned properties (for example, color or other attributes) and then use the spatial logic to reason about these properties. This would allow for a more expressive and flexible DSL, where the user can define complex spatial relationships and properties, and then use the spatial logic to reason about them.*

6.1 Currying of Functions

I also implemented the currying of functions, which allows for more expressive definitions.

For example, I can define a function that takes an angle and returns a function that takes a complex and rotates it by that angle:

```
function rotate_by(angle) {
    function rotate_complex(complex) {
        return rotate(complex, angle)
    }
    return rotate_complex
}

complex t1 = [p0, p1, p2]
complex t2 = rotate_by(90)(t1)
```

Listing 15: DSL Code for Example 3

7 Conclusion and Future Work

While the current implementation of the DSL establishes a foundational framework for intuitively defining and manipulating geometrical objects, there are several avenues for future enhancement.

The most important improvement would be to extend the DSL to add properties to the simplices, allowing users to assign attributes such as color. This would enable more complex spatial reasoning, as the spatial logic could then be used to query and manipulate these properties. For example, one could define the properties ”being red” and ”being blue” and check whether all the red simplices are connected to the blue ones.

Furthermore, another promising avenue for future research is to interface our simplicial representations with geometric model checking techniques designed specifically for continuous spaces [4]. This integration would pave the way for bridging the gap between discrete, combinatorial simplicial complexes and the analytical properties of continuous geometric structures.

Expanding the DSL or PolyLogicA to support algebraic topology concepts would actually be a significant enhancement. This would allow users to reason about holes, connected components, and other topological features of the simplicial complexes. For example, one could define the property ”having a hole” and check properties like ”all the red complexes have a hole” or ”there is a blue complex that is connected to a red complex with a hole”.

References

[1] Marco Aiello, Ian Pratt-Hartmann, and Johan van Benthem, *Handbook of Spatial Logic*, Springer, 2007.

[2] Vincenzo Ciancia, Diego Latella, Michele Loreti, and Mieke Massink, *Model Checking Spatial Logics for Closure Spaces*, *Logical Methods in Computer Science*, 12(4), 2016.

[3] Gina Belmonte, Vincenzo Ciancia, Diego Latella, and Mieke Massink, *VoxLogicA: A Spatial Model Checker for Declarative Image Analysis*, In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (TACAS), pp. 281–298, 2019.

[4] Nick Bezhanishvili, Vincenzo Ciancia, David Gabelaia, Gianluca Grilletti, Diego Latella, and Mieke Massink, *Geometric Model Checking of Continuous Space*, *Logical Methods in Computer Science*, 18(4), 2022.

[5] Gina Belmonte, Vincenzo Ciancia, and Mieke Massink, *Symbolic and hybrid AI for brain tissue segmentation using spatial model checking*, *Artificial Intelligence in Medicine*, 167: 103154, 2025.